

实验一 预备实验

练习二

【问题描述】

题目要求编写函数，统计数组 `a` 中所有奇数之和与所有偶数之和。题目给出的函数原型为 `void fun(int *a, int n, int *odd, int *even)`；实际代码在 Rust 中对应实现为 `fun(arr: &[i32], odd: &mut i32, even: &mut i32)`。主程序从 `inputfile.txt` 中读取整数数据，其中第一个整数为数组长度 `n`，后续 `n` 个整数为数组元素，程序调用 `fun` 后分别输出奇数和与偶数和。

本题的输入形式为文本文件输入，输入值类型为 32 位整数；输出形式为两行文本，分别给出“奇数之和”和“偶数之和”。系统应能正确处理普通数组、包含负数和 0 的数组，以及空数组切片等情况。

【基本思路】

存储结构上，主函数先把文件中的整数读入 `Vec<i32>` 动态数组，再根据第一个数取得有效数据区间，随后以切片 `&[i32]` 形式传入 `fun`。函数内部使用两个可变引用 `odd` 和 `even` 作为结果返回位置，这与原题“利用指针返回结果”的思路一致。

算法思想较为直接：先将奇数和、偶数和清零，再顺序遍历数组中的每一个元素；若当前元素对 2 取模后为 0，则累加到 `even`，否则累加到 `odd`。遍历结束后即可得到结果。该算法只需一次线性扫描，时间复杂度为 $O(n)$ ，额外空间复杂度为 $O(1)$ 。

```
main
  读取 inputfile.txt 中所有整数到 numbers
  n <- numbers[0]
  data <- numbers[1..=n]
  odd_sum <- 0, even_sum <- 0
  调用 fun(data, odd_sum, even_sum)
  输出 odd_sum 和 even_sum

fun(arr, odd, even)
  odd <- 0, even <- 0
  for num in arr
    if num % 2 == 0
      even <- even + num
    else
      odd <- odd + num
```

函数调用关系可表示为：`main()` → 读取文件数据 → `fun()` → 输出结果。程序主体负责输入输出，`fun` 只负责运算。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

- 1) 本部分采用 Rust 语言开发，代码组织为 Cargo 工程，核心函数位于 src/lib.rs，主程序位于 src/main.rs，另配有 tests/test_fn.rs 用于单元测试。主程序读取 inputfile.txt 中的数据；按照代码内容，示例输入为“5”和“1 2 3 4 5”，因此程序的输出应为：奇数之和 9，偶数之和 6。
- 2) 测试测试数据选取遵循“典型 + 边界 + 扩展”的原则：既验证正常混合数据，也验证负数、0 和空切片。测试重点在于元素分类逻辑是否正确，以及函数在不同输入规模下能否先清零再重新累加，避免旧值残留。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	n=5；数组为 [1, 2, 3, 4, 5]	奇数和=9，偶数和=6；与主程序示例输入一致。
2	数组为 [1, 2, 3, 4, 5, 6]	奇数和=9，偶数和=12；对应单元测试 mixed_numbers。
3	数组为 [-3, -2, -1, 0, 1, 2, 3]	奇数和=0，偶数和=0；说明程序能正确处理负数与 0。
4	空数组 []	奇数和与偶数和均重置为 0；验证边界条件处理正确。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于把“按奇偶分类统计”这一逻辑独立封装为函数，并正确使用可变引用返回两个结果；难点较小，主要在于文件读取后要准确切分出前 n 个有效数据，避免数组越界。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：结构简单、职责明确、时间复杂度低，而且测试覆盖了空数组与负数输入。可以改进的方向包括：在 main 中增加对 n 是否越界的显式判断；当输入文件格式错误时给出更明确的异常提示；支持从标准输入读取，以便程序在更多场景下复用。

3. 还有哪些实现方法？

可以不使用“先读全部再切片”的方式，而是边读取边统计，这样可以进一步减少主程序中的中间存储。

练习四

【问题描述】

题目要求编写函数，把数组 `a` 中的最大数和最小数交换。题目默认数组中无重复元素，对应函数原型为 `int exchange(int *a, int n)`；实际 Rust 代码实现为 `exchange(a: &mut [i32]) -> bool`。主程序从 `inputfile.txt` 中读取数据，第一个整数表示 `n`，后续 `n` 个整数为数组内容，调用 `exchange` 后输出交换后的数组。

输入形式仍然是文本文件输入，输出形式为一行交换后的数组，或在输入异常时给出提示信息。

【基本思路】

程序使用可变切片 `&mut [i32]` 作为待处理数组。若数组长度小于 2，则直接返回 `false`，表示无需交换；否则先假设第 0 个元素同时为最小值和最大值，再从下标 1 开始顺序扫描整个数组，分别记录当前最小值和最大值所在位置，最后调用 `swap(min_idx, max_idx)` 完成交换。

该算法同样只需要一次线性遍历，时间复杂度为 $O(n)$ ，额外空间复杂度为 $O(1)$ 。和题目中“无重复元素”的设定相比，实际代码做了更强的健壮性处理：即使存在重复最小值或重复最大值，程序仍然会交换首次出现的位置；若所有元素都相等，交换不会改变数组内容，但函数依然返回 `true`。

```
main
  读取 inputfile.txt 中所有整数到 numbers
  若 numbers 为空，输出“输入为空”
  n <- numbers[0]
  若 numbers.len < n + 1，输出“输入数据不足”
  data <- numbers[1..n]
  若 exchange(data) 为真，输出交换后的数组
  否则输出 err

exchange(a)
  if a.len < 2 return false
  min_idx <- 0, max_idx <- 0
  for i from 1 to a.len-1
    if a[i] < a[min_idx] then min_idx <- i
    if a[i] > a[max_idx] then max_idx <- i
  交换 a[min_idx] 和 a[max_idx]
  return true
```

函数调用关系可表示为：`main()` → 读取并校验输入 → `exchange()` → 输出数组。相比练习二，本题把“输入合法性判断”写得更完整，是一个较为规范的文件处理流程。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

- 1) 本部分使用 Rust 实现，主程序示例输入文件为“5”和“10 5 8 2 9”。按代码逻辑，原数组的最大值为 10、最小值为 2，交换后输出应为 [2, 5, 8, 10, 9]。
- 2) 测试数据覆盖普通情况、极值位于两端、空数组、单元素数组以及存在重复极值的情况。测试重点是 `min_idx` 和 `max_idx` 的定位是否正确，以及交换行为是否只影响两个目标位置。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	[10, 5, 8, 2, 9]	交换后为 [2, 5, 8, 10, 9]; 与 inputfile 示例一致。
2	[1, 3, 2, 9]	交换后为 [9, 3, 2, 1]; 验证极值位于两端时仍能正确处理。
3	空数组 []	返回 false, 数组保持为空; 验证边界输入。
4	[42]	返回 false, 数组保持不变; 说明长度不足 2 时不会误交换。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于一次遍历中同时维护最小值下标和最大值下标; 难点在于处理特殊输入时要保证程序行为清晰, 例如空数组、长度为 1 的数组、输入文件中 `n` 与数据数量不匹配的情况。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 主程序校验更完整, 核心函数逻辑简洁清晰, 并且通过单元测试主动覆盖了“重复极值”这一超出题目基础假设的情况。

3. 还有哪些实现方法?

可以先分别调用库函数求出最小值和最大值, 再查找其位置完成交换; 也可以在扫

描过程中把最小值、最大值和下标打包成一个结构体统一维护。

进阶练习二

【问题描述】

题目要求在二维平面上定义点和三角形的结构类型。已知三个点作为顶点构成一个三角形（面积 $S \geq 0$ ），程序需要完成以下任务：判断是否为等边三角形；判断是否为等腰三角形；检查是否为直角三角形；计算三角形面积；判断某一点是否位于三角形内部。实际代码中定义了 Point 结构和 Triangle 结构，并在 Triangle 上实现 area、is_valid、is_equilateral、is_isosceles、is_right、contains_point 等方法。

输入形式为文本文件中的 8 个浮点数，前 6 个数依次表示三角形三个顶点坐标 (x1, y1)、(x2, y2)、(x3, y3)，后 2 个数表示待检测点 P 的坐标。输出形式为五行文本，依次给出是否等边、是否等腰、是否直角、三角形面积以及点是否位于三角形内部。系统需要正确处理有效三角形，也要识别三点共线等无效情况。

【基本思路】

在存储结构上，Point 保存 x 和 y 两个 f64 浮点数；Triangle 由三个 Point 组成。为了减少开平方运算，程序用 side_lengths_sq() 统一返回三边长度的平方值，便于后续做等边、等腰和直角判定。由于浮点运算存在精度误差，代码额外设置 $EPS=1e-9$ ，并通过 approx_eq(a, b) 进行近似比较。

面积计算采用向量叉积公式： $S = |(B-A) \times (C-A)| / 2$ 。当面积小于等于 EPS 时，认为三点不能构成有效三角形。等边判定要求三边平方近似相等；等腰判定要求至少两边平方近似相等；直角判定先对三边平方排序，再检查“两短边平方和是否等于最长边平方”。点是否位于三角形内部则采用面积分割法：若以点 P 与原三角形三条边分别构成的三个小三角形面积之和等于原三角形面积，且三个小面积都严格大于 EPS，则认为点在三角形内部。由此可见，代码使用的是“严格内部”定义，边界点不算在内部。

```
读取 8 个浮点数
构造 A、B、C 三个点和点 P
triangle <- Triangle(A, B, C)
if triangle 无效
    输出 “三点不能构成有效三角形”
else
    输出 triangle.is_equilateral()
    输出 triangle.is_isosceles()
    输出 triangle.is_right()
    输出 triangle.area()
    输出 triangle.contains_point(P)
```

函数调用关系可表示为：main() → 构造 Point 与 Triangle → is_valid() →

is_equilateral()/is_isosceles()/is_right()/area()/contains_point() → 输出结果。其中 contains_point() 内部又会多次调用 area()。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

- 1) 主程序示例输入为“0 0 3 0 0 4 1 1”，对应三角形顶点 A(0,0)、B(3,0)、C(0,4)，待测点 P(1,1)。根据几何关系，该三角形不是等边三角形，也不是等腰三角形，是一个直角三角形，面积为 6.000000，点 P 位于三角形内部，因此程序的输出应分别为 false、false、true、6.000000、true。
- 2) 本部分测试数据设计比前两题更强调几何覆盖：既要验证等边、直角、一般三角形，也要验证内部点、外部点、边界点与退化三角形。测试重点在于浮点误差处理是否稳定，以及“严格内部”与“边界点不算内部”的判定是否一致。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	A(0,0), B(3,0), C(0,4), P(1,1)	非等边、非等腰、直角、面积 6、点在内部；与 inputfile 一致。
2	A(0,0), B(1,0), C(0.5, $\sqrt{3}/2$)	等边、等腰、非直角；验证等边判定。
3	A(0,0), B(4,0), C(0,3), P(1,1) / P(5,1) / P(2,0)	P(1,1) 在内部；P(5,1) 在外部；P(2,0) 在边界且返回 false。
4	A(0,0), B(1,1), C(2,2)	三点共线，面积为 0，判定为无效三角形。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于把几何概念转化为可复用的方法，并用结构体封装数据和行为；难点主要有两点：一是浮点数比较不能直接使用“==”，必须引入误差容忍；二是点在三角形内部的判定要先区分有效三角形，再明确边界是否计入内部。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：模块化程度高，几何意义清晰，side_lengths_sq() 的设计避免了重复计算和不必要的开平方；contains_point() 使用面积法，思路直观且易于验证。可改进方向包括：增加“点在边上”“点在顶点上”的单独判定接口；在输出中使用更友好的中文说明替代布尔值；进一步补充浮点极端数据测试。

3. 还有哪些实现方法?

直角判定也可以使用向量点积是否为 0; 点在三角形内部的判断也可使用重心坐标法或同向叉积法。个人认为当前实现比较适合作为预备实验, 因为它把结构体、方法、浮点误差处理和几何算法结合在一起, 既有基础性, 也有一定的综合训练价值。